

Voxel-Steered Autoregressive Generation: Overcoming the Quadratic Output Wall

Authors: Weslyn Cory Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: June 2026

Working Paper Series: AAMT-WP-21

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Status: Working paper, not peer reviewed. The Rolling Output Eviction loop and TERA-based output steering algorithms are implemented within the MaiiaM Alchemist engine (`apps/desktop/scripts/wp21_rolling_output_prototype.py`, `embedder_bridge.py`, `memory_index_bridge.py`). This paper extends **WP-20 (Voxel-Addressable Memory)** by inverting the retrieval mechanism from the input context space to the generative output space, yielding infinite effective output contexts at constant attention cost.

Abstract

Autoregressive language models face a strict quadratic wall: generation costs $O(N^2)$ in attention and $O(N)$ in Key-Value (KV) cache memory. To generate coherent long-form text (e.g., entire books), current models must hold the entire generated history in a dense KV cache. We present **Voxel-Steered Autoregressive Generation**, a geometric inversion of WP-20’s spatial index. By implementing a “Rolling Output Eviction” loop, the model evicts recently generated chunks from the dense KV cache into a persistent HNSW index over the TERA cube, querying its own past geometry to maintain coherence. Furthermore, by predicting coarse semantic cells (Meji/Odu) before decoding discrete tokens, the model generates a trajectory through meaning-space rather than a sequence of characters. We demonstrate that this architecture allows infinite-length coherent generation with constant $O(1)$ attention overhead and significant deduplication via Greedy Meshing.

1. The Asymmetry of Autoregression

The context limit of a Transformer applies identically to input prompts and generated outputs, but generation exacerbates the hardware penalty. To predict token $N + 1$, the model must attend to all N preceding tokens. Generating a 100,000-word novel token-by-token inflates the dense KV cache until the hardware runs out of memory.

Information theory indicates that retaining exact sequences of tokens is wildly inefficient for maintaining narrative or logical coherence. A human writing a novel does not hold the exact wording of chapter one in their working memory while writing chapter ten; they hold the *geometric attractor* (the plot, the archetype, the tone) and retrieve specifics on demand.

To break the autoregressive bottleneck, we must stop holding tokens and start indexing geometry.

2. Voxel Memory for Generative Output

WP-20 introduced a coarse-to-fine hierarchy over the unit TERA (Temporal, Emotional, Rational, Archetypal) cube: 1. **Meji (L1)**: 4-bit, 16 cells (coarse intent). 2. **Odu (L2)**: 8-bit, 256 cells (fine intent). 3. **Residual (L3)**: Whitened sentence embedding indexed via HNSW.

In WP-21, we apply this identical spatial index to the **output stream**.

2.1 Rolling Output Eviction (Tier 0 to Tier 1)

Instead of a monolithic KV cache that grows linearly with generation, we define: - **Tier 0 (Working Set)**: A strict sliding window of the last k generated tokens (e.g., $k = 512$). - **Tier 1 (Geometric Store)**: The HNSW index of all previously generated text chunks.

When a semantic chunk (a sentence or paragraph) passes out of the Tier 0 window, it is not discarded. It is embedded, whitened, and stored in the Tier 1 Voxel space.

As the model generates new text, the current thought acts as a query to the Voxel space. The engine retrieves the k -nearest geometrically similar past thoughts and injects them into the prompt. The effective generation context becomes infinite ($N \rightarrow \infty$) while the active KV cache remains strictly bounded ($O(1)$).

2.2 Hierarchical Generation (Coarse-to-Fine Decoding)

Standard Transformers predict discrete tokens directly from a vast vocabulary space ($\approx 50,000$ tokens). Voxel-steered generation separates semantic planning from token decoding:

1. **Predict Meji**: The model predicts the next 4-bit Meji coordinate, determining the structural intent of the next paragraph.
2. **Predict Odu**: The model refines the 4-bit coordinate to an 8-bit Odu target.
3. **Decode Tokens**: A local decoder conditioned on the Odu spatial coordinate fills in the discrete tokens.

The generation becomes a trajectory over a 4D manifold. We are no longer generating tokens; we are steering through meaning-space.

3. Greedy Meshing for Output Deduplication

A major inefficiency in long-form generation (especially code generation) is the repetition of identical semantic structures. If the model generates standard boilerplate or repeated logical arguments, it wastes compute recalculating the attention matrices for identical concepts.

Applying the **Greedy Meshing** principle from 3D voxel rendering to the generative output: - When the model’s intended trajectory nears a Tier 1 attractor that is highly similar (cosine > 0.95) to a previously generated chunk, it does not decode the tokens. - Instead, it outputs the **sparse chunk address** (the Morton key) of that geometric cell. - The runtime engine interpolates the cached text into the output stream directly.

This literal Hopfield consolidation prevents the generative model from repeating itself and strips out redundant compute.

4. Architecture Implementation (The Rolling Loop)

The core loop implemented in the MaiaM engine (`wp21_rolling_output_prototype.py`) demonstrates this in Python:

```

# The Rolling Output Eviction Loop
while True:
    # 1. Query Voxel Space for relevant past thoughts
    past_thoughts = memory_index.query(current_thought_embedding, k=3)

    # 2. Inject into Tier 0 Working Set
    prompt = build_prompt(past_thoughts, recent_context)

    # 3. Generate Next Chunk (Tier 0)
    generated_chunk = llm.generate(prompt)

    # 4. Evict to Tier 1
    chunk_embedding = embedder.embed(generated_chunk)
    memory_index.add(chunk_embedding, generated_chunk)

    # 5. Slide Window
    update_recent_context(generated_chunk)

```

By decoupling the memory state from the attention matrix, the model can generate a “massive continuous log” on deep, sprawling topics (e.g., consciousness, wisdom, philosophy) without ever losing coherence or hitting an out-of-memory error.

5. Conclusion

Voxel-Steered Autoregressive Generation (WP-21) demonstrates that the same geometric spatial indices that solve input context limits can solve the generative output wall. By establishing a Rolling Output Eviction pipeline and utilizing coarse-to-fine TERA prediction, we enable generative AI to write infinitely long, deeply coherent sequences at a constant processing cost. The bottleneck of sequence generation is not resolved by massive compute clusters, but by the application of fundamental spatial geometry.